Stripe billing health check

Generated by `health-check` from stripe-billing-reliability-kit. Read-only: nothing was changed.

Severity	Count
Critical	0
High	4
Medium	2
Low	1

Code

Scanned 1 file. Webhook handler file(s): `server.js`.

1. Signature is checked against a parsed (re-serialised) body

🟠 **High** · definite · `server.js:14` · rule H2

```
event = stripe.webhooks.constructEvent(JSON.stringify(req.body), sig,
process.env.STRIPE_WEBHOOK_SECRET);
```

Why it matters. express.json()/bodyParser has already parsed the body, so the bytes no longer match what Stripe signed. Verification fails at random; teams often 'fix' it by turning verification off.

Fix. Mount express.raw({ type: 'application/json' }) on the webhook route only (before any global JSON parser) and pass that Buffer to constructEvent.

2. No sign of duplicate-event protection

🟠 **High** · likely, needs a human look · `server.js:20` · rule H4

Why it matters. Stripe delivers events at least once. Without recording event.id, a retry or resend runs side effects again: double credits, duplicate emails, two provisioning runs.

Fix. Insert event.id into a table with a unique key (ON CONFLICT DO NOTHING) in the same transaction as the side effect, and skip events already recorded.

3. Access is granted from the checkout success redirect

🟠 **High** · likely, needs a human look · `server.js:32` · rule H5

```
await db.users.update({ where: { email: session.customer_details.email }, data: { plan: "pro" } });
```

Why it matters. The success URL can be reopened, shared or reached when payment later fails (3-D Secure, async methods). Access and Stripe drift apart.

Fix. Let only the verified webhook (or reconcile) grant access. The success page should just poll your own server state until the webhook has run.

4. Cancellations are never handled (no customer.subscription.deleted)

🟠 **High** · definite · `server.js:20` · rule H6

Why it matters. Customers who cancel, or whose payments fail until Stripe gives up, keep paid access forever.

Fix. Handle customer.subscription.deleted (and .updated) and recompute access from the subscription status.

5. Plan changes and status changes are ignored (no customer.subscription.updated)

🟡 **Medium** · definite · `server.js:20` · rule H6

Why it matters. Upgrades, downgrades, past_due and cancel-at-period-end never reach your database.

Fix. Handle customer.subscription.updated by re-syncing the subscription.

6. Failed renewals are not handled (no invoice.payment_failed)

🟡 **Medium** · definite · `server.js:20` · rule H6

Why it matters. You can't warn the customer or apply a grace period, so access either continues unpaid or stops with no explanation.

Fix. Handle invoice.payment_failed: re-sync the subscription (it becomes past_due) and notify the customer.

7. No reconciliation job (nothing lists subscriptions from Stripe)

⚪ **Low** · likely, needs a human look · `server.js:20` · rule H9

Why it matters. If a webhook is never delivered (endpoint down, deploy, misconfiguration), a paying customer stays without access until they complain.

Fix. Run a scheduled job that lists subscriptions from Stripe and repairs any that differ from your database, through the same code path as webhooks.

Automated checks catch the common, expensive mistakes. They can miss logic that is spread across files or written in an unusual way, and a "likely" finding can be a false alarm. Each rule maps to a mechanism in this kit, tested against real Stripe test mode: see the repository README.